

Medicine Directory

Software Engineering Intern (Web Platform) — Take-Home Assignment

Candidate Reference Guide

Quick Facts

Timeline: **5 Aug 2026 - 4PM**

Stack: **React/Next.js + TypeScript + Tailwind (SEO - FRIENDLY WEB PLATFORM)**

No backend, no API key needed

AI use is allowed — but **full prompt history must be submitted**

ASSIGNMENT LINK :-  [Take-Home Assignment - 2026_ NxtWave_ Software Engineering Intern \(Web Platform\).pdf](#)

GOOGLE FORM SUBMISSION LINK :-

1. Objective

Build a Medicine Directory web app on top of the public openFDA Drug Label API. The app has two core pages: a homepage where a user searches for a medicine by name, and a medicine details page showing active ingredients, purpose, warnings, and dosage.

The brief deliberately leaves *"SEO-friendly"* undefined for the details page — you decide what that means and design for the **Indian market**: who is landing on the page, what device they're likely on, what layout builds trust, and what would actually help the page rank. Your reasoning matters as much as the output.

What you'll practice:

- Consuming and typing a real, imperfect public API response
- Next.js/React routing and on-page SEO fundamentals (metadata, semantic HTML, URL structure)
- Designing visible UI states for edge cases, not just try/catch handling
- Writing clean, duplication-free components with Tailwind used intentionally
- Working transparently with AI coding tools and documenting your prompt history

2. Technical Requirements

- **React.js or Next.js** — your framework. Next.js is a natural fit since it gives file-based routing and metadata handling for the SEO-friendly details page.
- **TypeScript (required)** — type the openFDA response so the rest of the app is built on safe, predictable data.
- **Tailwind CSS (required)** — for styling, used with intent (composed utilities, consistent spacing) rather than fought against.
- **TanStack Query** — optional bonus; useful for fetching, caching, and built-in loading/error states.
- **No backend** — all data comes directly from the openFDA API, called client-side or from a Next.js server component/route handler.
- **openFDA Drug Label API** — public, no API key needed. Docs: open.fda.gov/apis/drug/label. Test query:

```
GET https://api.fda.gov/drug/label.json?search=openfda.brand_name:"advil"&limit=5
```

- **AI coding harness (mandatory if AI is used)** — Claude Code, OpenCode, Roo Code, Cursor, VS Code Copilot (Chat/Agent mode), or Windsurf/Devin — these retain a session log that becomes part of your submission.

3. Project Features

Homepage

- A search input where a user types a medicine name
- Calls the openFDA API and lists matching results
- Visibly handles zero results and ambiguous multi-match searches (e.g. one brand, several formulations)

Medicine Details Page

- A dedicated route per medicine, e.g. /medicine/[name]
- Shows active ingredients, purpose, warnings, and dosage
- SEO-friendly by your own definition — be ready to justify the choice
- Consider whether a health-information disclaimer belongs on the page

4. Step-by-Step Implementation Guide

1. **Explore the API.** Run the test query and inspect the JSON — note optional fields, nested arrays, and how multiple formulations of one drug appear. You can't design the types or the UI correctly until you know the real shape of the data.
2. **Set up the project.** Scaffold React or Next.js with TypeScript and Tailwind (add TanStack Query if you're using it). This gives you a typed, styled foundation to build on.
3. **Type the API response.** Write TypeScript interfaces for the openFDA label object before writing UI code, so missing or optional fields are a deliberate decision, not a surprise crash.
4. **Build the homepage search.** Add the search input, call the API, and render the matches — including the zero-result and multi-match states, in the UI itself.
5. **Build the medicine details page.** Create the dynamic route, fetch and display the ingredient/purpose/warning/dosage fields, and implement your SEO decisions (metadata, URL, semantic HTML, mobile-first layout).
6. **Handle edge cases visibly.** No results, API error/timeout, and missing fields should each render a real UI state — not just be caught in a try/catch and ignored.
7. **Review code quality.** Remove duplicated logic, keep components small and single-purpose, and check that Tailwind classes are composed with intent.
8. **Write the README and export your prompt history.** Document your decisions and, if you used AI, export the full raw session log to prompt-history.md before you submit.

5. Project Structure

```
medicine-directory/
├── app/
│   ├── page.tsx           # Homepage – search UI
│   ├── layout.tsx        # Root layout, global metadata
│   └── medicine/
│       └── [name]/
│           └── page.tsx    # Medicine details page (SEO route)
└── components/
    └── SearchBar.tsx      # Homepage search input
```

— MedicineCard.tsx	# Result list item
— ErrorState.tsx	# Shared no-results / error / timeout UI
— lib/	
— openfda.ts	# API client
— types/	
— medicine.ts	# TypeScript interfaces for API data
— prompt-history.md	# Full raw AI prompt history (if AI used)
— README.md	# Decisions, trade-offs, time spent
— package.json	

- **app/** — routes and pages, including the dynamic SEO-friendly medicine route.
- **components/** — small, reusable UI pieces, including a shared component for error/empty states.
- **lib/ + types/** — the API client and its TypeScript types, kept separate from UI code.

6. Example Code Snippets

Typing the openFDA response

```
interface DrugLabel {
  openfda: {
    brand_name?: string[];
    generic_name?: string[];
  };
  purpose?: string[];
  warnings?: string[];
  dosage_and_administration?: string[];
}
```

Calling the API for a search term

```
const res = await fetch(
  `https://api.fda.gov/drug/label.json?search=openfda.brand_name:"${term}"&limit=5`
);
if (!res.ok) throw new Error("openFDA request failed");
const data = await res.json();
```

Always check res.ok before parsing — the API can return non-200 responses that shouldn't be treated as valid data.

A visible error state

```
{status === "empty" && <p>No medicines matched "{term}"</p>}
{status === "error" && <p>Something went wrong. Please try again.</p>}
```

Edge cases should be states the UI renders, not just errors swallowed in a catch block.

7. Error Handling Guidelines

Handle these visibly in the UI — not only inside a try/catch:

- **No results found** — show a clear message, not a blank page.
- **API error or timeout** — show a retry-friendly message, not a crash or an infinite spinner.
- **Missing fields on a result** — degrade gracefully (e.g. "Dosage not listed") instead of showing "undefined".

8. AI Tool Usage & Prompt History

AI use is fine — expected, given the 24-hour window — but it must go through a harness that keeps a session log: Claude Code, OpenCode, Roo Code, Cursor, VS Code Copilot (Chat/Agent mode), or Windsurf/Devin.

Submit your full raw prompt history, in order — every prompt, not a summary — as `prompt-history.md`. Example format:

```
[Turn 1] "Set up a Next.js app with TypeScript and Tailwind, App Router"
[Turn 2] "Add a search bar on the homepage that calls the openFDA label API"
[Turn 3] "This is returning weird nested arrays for some drugs, why"
[Turn 4] "That fallback UI looks off, redo it without a full-page spinner"
```

Export tips (exact steps vary by tool version — check your tool's docs if these don't match):

- **Claude Code:** session logs live on disk, typically under a project-local or `~/.claude` history folder. Include the transcript, or use your version's `export/transcript` command.
- **Cursor:** open the chat/Composer history for the session and export or copy the full thread.
- **VS Code Copilot Chat:** open the Chat panel history and export or copy it.
- **Windsurf, Roo Code, OpenCode, Devin:** each has a session history in its chat panel — find the export option. No clean export? Keep a running `.txt/.md` log as you go instead.

Didn't use AI? Just say so in your README.

9. Evaluation Criteria

- **Product thinking** — deliberate calls on layout, content, and SEO for the Indian context (not a generic template), how you handle multiple formulations of the same drug, and whether you show a health disclaimer.
- **Edge cases** — zero results, ambiguous multi-match searches, missing fields, slow requests, malformed responses.
- **Code quality** — no duplicated logic, clean components, Tailwind used properly — not just made to work.
- **Thinking process** — visible in your prompt history: what you researched, what you rejected and why, how fast you iterated.

No one nails all four. The evaluators are looking for where you spike, and how much care went in.

10. Submission Guidelines

Submit the following:

1. **Public GitHub repository link** (or a zip) with the complete project source code.
2. **prompt-history.md** with your full raw AI prompt history, if AI tools were used.
3. **README.md** covering: your product/SEO/layout decisions (especially anything India-specific), what you'd do differently with more time, and the time spent plus which AI harness, if any, you used.

Timeline: 24 hours from receipt.